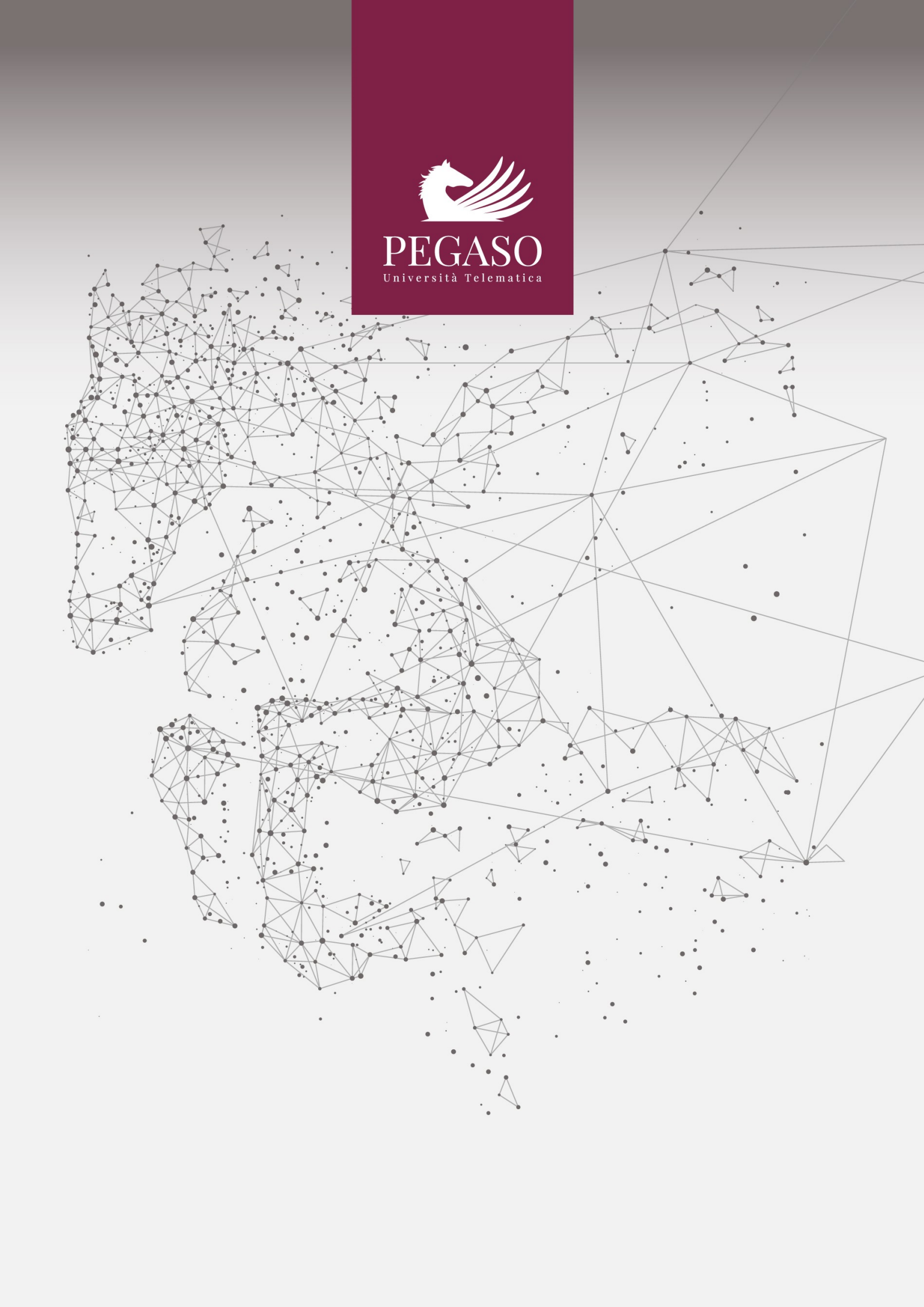

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. SOMMA E MEDIA DI UNA LISTA....4
3. INSERIMENTO ORDINATO IN UNA LISTA....6
4. SLIDING WINDOW....9
5. SLIDING WINDOW DI DIMENSIONE N....12
BIBLIOGRAFIA....15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano ed analizzano alcune esercitazioni molto importanti che approfondiscono le operazioni di inserimento e di elaborazione delle liste. In particolare, si riportano quattro esercizi in cui si risolvono tipici problemi relativi alle liste. Il primo esercizio mostra funzioni di caricamento e di elaborazione (somma e media degli elementi) della lista. Il secondo esercizio mostra come operare l'inserimento di un elemento in una lista ordinata al fine di mantenere l'ordinamento (crescente o decrescente) della lista stessa. Il terzo e il quarto esercizio illustrano un esercizio tipico di elaborazione di liste numeriche, l'esercizio dello sliding window (finestra scorrevole), rispettivamente per una larghezza pari a tre e per una larghezza pari a n (con n dispari) della finestra scorrevole.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Somma e media di una lista

L'esercizio proposto prevede il calcolo della somma e della media di una lista. L'algoritmo per svolgere l'esercizio prevede la suddivisione in funzioni per:

- caricare la lista con elementi a scelta dell'utente – funzione caricavettore();
- sommare tutti gli elementi della lista – funzione somma();
- calcolare la media degli elementi della lista – funzione media().

Il caricamento del vettore avviene tramite inserimento di un elemento della lista e successiva risposta positiva/negativa alla volontà di inserire ulteriori elementi attraverso una struttura iterativa. Analizziamo le funzioni necessarie per la risoluzione dell'esercizio:

- funzione caricavettore(): non richiede parametri e restituisce una lista. Il caricamento del vettore avviene tramite inserimento di un elemento della lista e successiva risposta positiva/negativa alla volontà di inserire ulteriori elementi attraverso una struttura iterativa;
- funzione somma(v): accetta una lista come parametro e restituisce un numero intero. Inizializzando a zero una variabile accumulatore s, si scorrono tutti gli elementi della lista e si sommano ad s;
- funzione media(v): accetta una lista come parametro e restituisce un numero decimale. La media viene calcolata richiamando la funzione somma() e dividendo il risultato restituito dalla funzione per la dimensione della lista len(v);
- funzione main(): richiama la funzione caricavettore() per ottenere il vettore, stampa il risultato delle funzioni somma(v) e media(v).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def caricavettore():
    v = []
    r = 's'
    while r=="s" or r=="S":
        a = int(input("Inserire numero: "))
        v.append(a)
        r = input("Vuoi inserire un altro numero? (S/N) ")
    return v
def somma(v):
    s = 0
    for elem in v:
        s+=elem
    return s
def media(v):
    return float(somma(v))/len(v)
def main():
    print("SOMMA MEDIA LISTA")
    v = caricavettore()
    print("La somma dei valori della lista è", somma(v))
    print("La media dei valori della lista è", media(v))


main()
```

```
SOMMA MEDIA LISTA
Inserire numero: 4
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 5
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 6
Vuoi inserire un altro numero? (S/N) n
La somma dei valori della lista è 15
La media dei valori della lista è 5.0
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Inserimento ordinato in una lista

Obiettivo dell'esercizio è quello di operare l'inserimento di elementi numerici interi in una lista sempre ordinata senza utilizzare funzioni speciali built-in di Python. L'algoritmo procede nel seguente modo:

- partendo da una lista vuota, l'elemento che si inserisce può essere aggiunto alla lista vuota in append;
- una volta inserito il primo elemento, se il nuovo elemento è maggiore di quello già presente viene posizionato in coda alla lista in append;
- in caso contrario (nuovo elemento minore dell'elemento già presente), il nuovo elemento viene inserito in prima posizione e l'elemento già presente viene spostato di una posizione verso destra;
- generalizzando, per una lista ordinata di lunghezza n:
    - l'inserimento del nuovo elemento prevede lo scorrimento della lista fino all'individuazione della posizione corretta *i* nella lista dove inserire il nuovo elemento al fine di mantenere l'ordinamento;
    - lo spostamento (shifting) di una posizione verso destra di tutti gli elementi presenti nelle posizioni più a destra di *i*;
    - inserimento del nuovo elemento nella posizione corretta *i*.

Nella figura seguente si riporta l'esempio della lista [1,2,4,5,6] in cui si vuole inserire il numero 3 mantenendo ordinata la lista. Individuato il punto corretto per l'inserimento (indice = 2), si procede con lo slittamento della sotto-lista [4,5,6] di una posizione verso destra e l'inserimento del numero 3 nella posizione avente indice 2.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice Python che segue riporta sia la funzione main(), sia la funzione ins_ordinato(). La funzione main() provvede all'input degli elementi della lista da parte dell'utente e richiama la funzione ins_ordinato() per il vero e proprio inserimento ordinato di ciascun elemento all'interno della lista.

La funzione ins_ordinato() accetta come parametri la lista v e l'elemento da inserire *a* e opera nel seguente modo:

- verifica che la lunghezza della lista sia zero o che l'elemento da inserire sia maggiore dell'ultimo elemento della lista; in tale caso, l'elemento da inserire nella lista viene posto in coda alla lista stessa;
- in caso contrario, l'algoritmo scorre la lista per trovare la corretta collocazione (valore dell'indice) in cui l'elemento dovrà essere inserito;
- una volta trovata tale posizione, avviene lo slittamento (shifting) di tutti gli elementi a destra di tale collocazione. Lo shifting avviene avendo cura di spostare di una posizione a destra ciascun elemento della sotto-lista da far slittare, partendo da destra (ultimo elemento) verso sinistra al fine di evitare la sovrascrittura degli elementi della lista per lo shifting;
- al termine dello shifting, si inserisce nella corretta collocazione il nuovo elemento che si voleva inserire nella lista.

```
def ins_ordinato(v,a):
    if len(v)==0 or v[-1]<a:
        v.append(a)
    else:
        i=0
        while a>v[i] and i<len(v)-1:
            i+=1
        v.append(v[-1])
        for j in range(len(v)-2,i-1,-1):
            v[j+1]=v[j]
        v[i]=a
    print("dopo l'inserimento di",a,"v=",v)
    return v
def main():
    print("INSERIMENTO ORDINATO LISTA")
    v=[]
    r = 's'
    while r=="s" or r=="S":
        a = int(input("Inserire numero: "))
        v = ins_ordinato(v,a)
        r = input("Vuoi inserire un altro numero? (S/N) ")

    print("Lista ordinata:",v)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
main()

INSERIMENTO ORDINATO LISTA
Inserire numero: 1
dopo l'inserimento di 1 v= [1]
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 8
dopo l'inserimento di 8 v= [1, 8]
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 6
dopo l'inserimento di 8 v= [1, 6, 8]
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 3
dopo l'inserimento di 8 v= [1, 3, 6, 8]
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 2
dopo l'inserimento di 8 v= [1, 2, 3, 6, 8]
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 22
dopo l'inserimento di 22 v= [1, 2, 3, 6, 8, 22]
Vuoi inserire un altro numero? (S/N) s
Inserire numero: 22
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

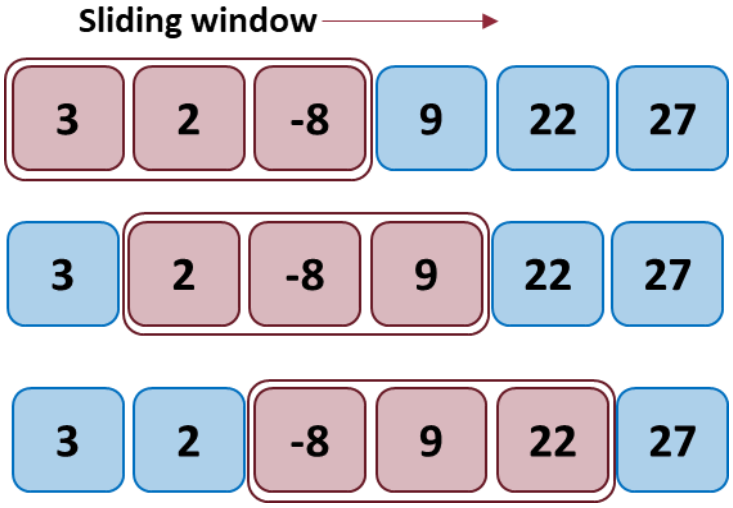
4. Sliding window

L'esercizio della finestra scorrevole (sliding window) è un esercizio che ha diversi usi nel campo dell'informatica, delle telecomunicazioni e dell'elaborazione dei segnali. Un esempio di applicazione dell'algoritmo dello sliding window può essere lo smoothing (in italiano «addolcimento») di segnali rumorosi.

Tale algoritmo opera nel seguente modo:

- si considera una finestra virtuale di larghezza n (es. n=3) che scorre di una posizione verso destra ad ogni iterazione;
- in una nuova lista, il primo elemento sarà uguale al risultato di un'operazione (es. media) applicata a tutti gli elementi contenuti nella finestra;
- all'iterazione successiva la finestra si sposta di una posizione a destra, si procede di nuovo con l'operazione sugli elementi all'interno della finestra e si ottiene il secondo elemento della nuova lista;
- si procede finché il «bordo» destro della sliding window non raggiunge il termine della lista.

Nella figura successiva si riporta uno schema dello spostamento della sliding window per una determinata lista. A ciascuna iterazione, la finestra scorrevole si sposta di una posizione verso destra.



È facilmente intuibile che la nuova lista che si genera conterrà meno elementi della lista di partenza. Più precisamente la lunghezza della nuova lista sarà pari alla lunghezza della lista originale meno la larghezza della finestra scorrevole più uno (es. len(lista) = 6; window=3; len(nuova_lista) = 6-3+1 = 4.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel programma che andremo a codificare, si impone una lunghezza della sliding window pari a 3 e la media come operazione da effettuare sugli elementi della finestra. Si utilizzerà la funzione carica_vettore() per caricare la lista su cui applicare l'algoritmo.

La funzione sliding window() permetterà l'elaborazione della nuova lista in base all'algoritmo appena analizzato. L'impostazione del range della struttura iterativa è molto semplice in questo caso poiché l'indice corrisponderà al centro della finestra e nella media verranno coinvolti, oltre all'elemento centrale della finestra, l'elemento a sinistra e l'elemento a destra dello stesso.

Di seguito si riporta il codice Python che implementa le funzioni caricavettore(), sliding window() e il main(). Si noti che la struttura iterativa nella funzione sliding_window() avendo una larghezza della finestra pari a 3 permetterà alla finestra di:

- partire in posizione 1: finestra centrata sulla seconda posizione per permettere l'estensione della finestra dalla posizione 0 alla posizione 2;
- terminare in posizione len(v)-2: finestra centrata sulla penultima posizione per permettere l'estensione della finestra dalla posizione len(v)-3 alla posizione len(v)-1.

```
def caricavettore():
    v = []
    r = 's'
    while r=="s" or r=="S":
        a = int(input("Inserire numero: "))
        v.append(a)
        r = input("Vuoi inserire un altro numero? (S/N) ")
    return v
def sliding_window(v):
    sw = []
    for i in range(1,len(v)-1):
        sw.append((v[i-1]+v[i]+v[i+1])/3.0)
    return sw
def main():
    print("SLIDING WINDOW")
    #v = caricavettore()
    v=[1,2,3,4,5,6,7,8,9]
    print("v=",v)
    if len(v)>=3:
        sw = sliding_window(v)
        print("Il risultato dell'applicazione della sliding window
di larghezza 3 è")
        print(sw)
    else:
        print("Lunghezza della lista inferiore a 3, sliding window
non applicabile")

main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il risultato dell'esecuzione del codice Python della sliding window con larghezza pari a 3 alla lista 1,2,3,4,5,6,7,8,9.

```
SLIDING WINDOW
v= [1, 2, 3, 4, 5, 6, 7, 8, 9]
Il risultato dell'applicazione della sliding window di larghezza 3 è
[2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]
```

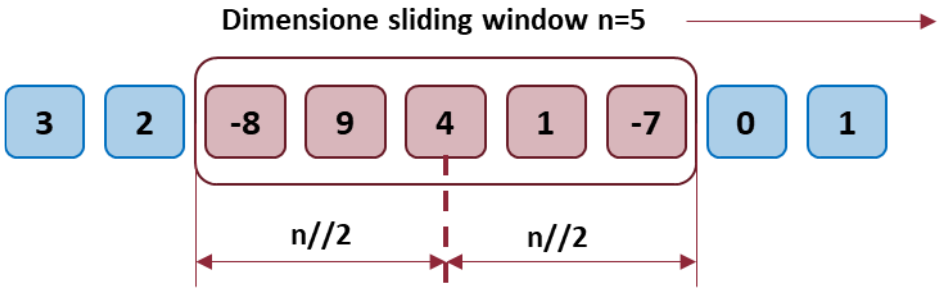
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Sliding window di dimensione n

L'esercizio che si propone è una generalizzazione dell'algoritmo sliding window con larghezza della finestra pari a 3 proprio perché si vuole ottenere il codice per una larghezza della finestra pari a n, con n dispari.

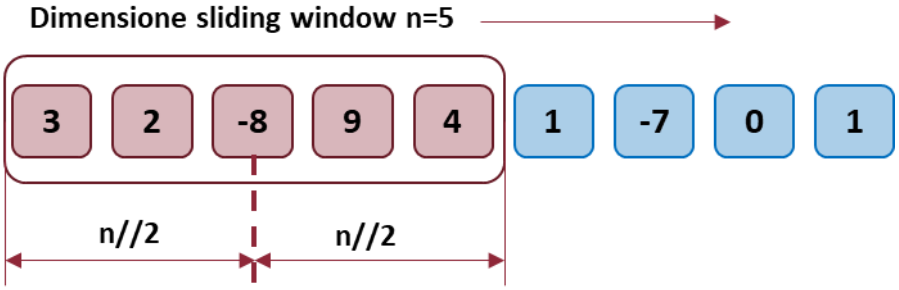
Il codice visto in precedenza deve essere modificato innestando nella struttura iterativa che «scorre» la lista, un'ulteriore struttura iterativa che «scorre» la finestra di lunghezza n. In particolare, la finestra dovrà essere scorsa da n//2 posizioni precedenti al centro della finestra a n//2 posizioni dopo il centro della finestra stessa come schematizzato nella figura seguente.



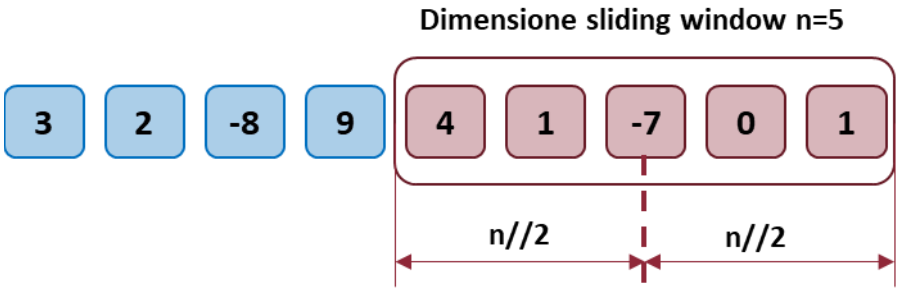
Ad ogni modo nella struttura iterativa esterna, è necessario considerare anche la dimensione della finestra per calcolare il range entro cui far variare il centro della finestra. Poiché la finestra occuperà n//2 posizioni prima del centro della finestra stessa e n//2 posizioni dopo il centro della finestra, l'intervallo di variazione dell'indice rappresentante il centro della finestra dovrà essere:

- da n//2 (indice delle liste inizia da zero);
- a n-n//2;

come si può osservare, rispettivamente, dai due schemi riportati di seguito.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



La funzione main() rimane praticamente identica al caso per larghezza della sliding window pari a 3 con la sola aggiunta:

- di un controllo sulla dimensione della sliding window n rispetto alla dimensione della lista len(v) (l'algoritmo si può applicare solo se len(v)≥n);
- alla funzione sliding_window() andrà passata come parametro anche la dimensione della finestra n.

```
def main():
    print("SLIDING WINDOW")
    v = caricavettore()
    #v=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
    n=2
    while n%2==0:
        n = int(input("Inserire dimensione sliding window
(numero dispari): "))
    print("v=",v)
    if len(v)>=n:
        sw = sliding_window(v,n)
        print("Il risultato dell'applicazione della sliding window di
larghezza",n,"è una lista di lunghezza",len(sw))
        print(sw)
    else:
        print("Lunghezza della lista inferiore a",n,"- Sliding window
non applicabile")


main()

def caricavettore():
    v = []
    r = 's'
    while r=="s" or r=="S":
        a = int(input("Inserire numero: "))
        v.append(a)
        r = input("Vuoi inserire un altro numero? (S/N) ")
    return v
def sliding_window(v,n):
    sw = []
    for i in range(n//2,len(v)-n//2):
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
        somma=0
        for j in range(i-n//2,i+n//2+1):
            somma+=v[j]
        sw.append(somma/float(n))
    return sw
```

Di seguito si riporta il risultato dell'esecuzione del codice Python della sliding window con larghezza pari a n, con n dispari per tre casi diversi:

1. lista [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15] con dimensione della sliding window n=5;
2. lista [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15] con dimensione della sliding window prima pari a 16 (valore non valido poiché pari) e poi pari a 17;
3. lista [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15] con dimensione della sliding window n=11.

```
SLIDING WINDOWS
Inserire dimensione sliding window (numero dispari): 5
v= [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
Il risultato dell'applicazione della sliding window di larghezza 5
è una lista di lunghezza 11
[3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0]
>>>
SLIDING WINDOW
Inserire dimensione sliding window (numero dispari): 16
Inserire dimensione sliding window (numero dispari): 17
v= [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
Lunghezza della lista inferiore a 17 - Sliding window non applicabile
>>>
SLIDING WINDOW
Inserire dimensione sliding window (numero dispari): 11
v= [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
Il risultato dell'applicazione della sliding window di larghezza 11
è una lista di lunghezza 5
[6.0, 7.0, 8.0, 9.0, 10.0]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).